

Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA152 Microcontrollers

Last Name: LIBRE

Date: 09/24/2026

First Name: MARK LOUIE

Subject/Section: BCA152 – B86.1

Laboratory Report
Real-Time Multisensor Room Monitoring System

1. Problem and Requirements

1.1 Problem

The laboratory activity required the development of a real-time multisensor room monitoring system using an ESP32 microcontroller and FreeRTOS. The system monitors environmental conditions and room activity and presents the information to the user through an OLED display.

The system must simultaneously handle temperature and humidity measurement, ambient-light measurement, motion detection, user navigation, display management, and temperature alarm control. These functions must operate as separate concurrent activities rather than being implemented as one large sequential program.

The main software-engineering problem is therefore to coordinate multiple tasks while maintaining predictable timing, safe communication, and proper synchronization between shared resources.

1.2 Required System Behavior

The system is required to:

- Measure temperature using a DHT22 sensor.
- Measure humidity using a DHT22 sensor.
- Measure relative ambient light using an LDR and ADC.
- Detect motion using a PIR sensor.
- Display the selected measurement on an SSD1306 OLED.
- Allow the user to navigate through Temperature, Humidity, Light, and Motion using a rotary encoder.
- Activate a buzzer when the temperature is outside the configured normal range.

- Maintain ACTIVE and INACTIVE operating states.
- Enter the INACTIVE state after a period without motion.
- Return to ACTIVE when motion is detected while INACTIVE.

The laboratory specifies a normal temperature range of **18 °C to 30 °C**. Temperatures below 18 °C are considered low, temperatures from 18 °C through 30 °C are normal, and temperatures above 30 °C are high.

1.3 Wokwi Adaptation

The required hardware was adapted to the Wokwi simulation environment. The simulated system consists of an ESP32 DevKit C V4, DHT22 sensor, LDR, PIR sensor, KY-040 rotary encoder, SSD1306 OLED, and buzzer.

The main connections used were:

Component	ESP32 Connection	Function
DHT22	GPIO 4	Temperature and humidity
LDR	GPIO 34 / ADC	Relative light level
SSD1306 OLED	SDA GPIO 21, SCL GPIO 22	Display
Rotary Encoder	CLK GPIO 18, DT GPIO 19, SW GPIO 5	Navigation
PIR	GPIO 27	Motion detection
Buzzer	GPIO 27	Alarm output

Wokwi provides a controlled simulation environment that allows the firmware, FreeRTOS architecture, sensor interactions, user input, and state transitions to be tested without requiring physical hardware.

2. System Architecture and Design

2.1 Hardware Architecture

The ESP32 functions as the central controller. Sensor inputs are connected to GPIO and ADC interfaces, while the OLED uses I2C communication. The rotary encoder provides user input, the PIR provides motion information, and the buzzer provides an audible alarm output.

The DHT22 supplies temperature and humidity values. The LDR is connected to an ADC input and produces a relative light-level value. The PIR provides a digital motion signal.

The OLED is used as the main user interface. Only one selected measurement is displayed at a time.

2.2 Software Architecture

The software is divided into independent modules according to their responsibilities. The major modules are:

- Sensor acquisition
- Display management
- Rotary encoder input
- Motion detection
- Alarm evaluation
- Alarm output
- System-state management
- FreeRTOS object management

The ESP-IDF application entry point initializes the required hardware and FreeRTOS objects before creating the application tasks.

This organization keeps hardware-specific operations and application logic separated as much as practical and allows deterministic logic such as alarm evaluation and state transitions to be tested independently.

2.3 Subsystem Decomposition

Subsystem	Responsibility
Sensor subsystem	Reads temperature, humidity, and relative light level
Display subsystem	Controls the SSD1306 OLED
Input subsystem	Processes rotary encoder input
Motion subsystem	Processes PIR input and manages activity state
Alarm subsystem	Evaluates temperature and controls the buzzer
RTOS object subsystem	Provides queues, event groups, mutexes, and shared RTOS resources

The SensorData structure is used to represent the current sensor information:

- Temperature
- Humidity
- Light level
- Motion detection status

This structure provides a consistent format for communication between tasks.

2.4 State Machine

The system uses two main activity states:

ACTIVE

The system is monitoring the room and displaying the selected measurement.

INACTIVE

The system has detected no motion for the configured inactivity period. The OLED is cleared while the system waits for new motion.

The primary transitions are:

- ACTIVE → INACTIVE when the inactivity timeout expires.
- INACTIVE → ACTIVE when PIR motion is detected.
- Motion while ACTIVE resets the inactivity timer and keeps the system ACTIVE.

For laboratory testing, the inactivity timeout was configured to approximately **15 seconds**.

3. FreeRTOS Architecture

The application uses five primary FreeRTOS tasks:

1. SensorTask
2. DisplayTask
3. InputTask
4. MotionTask
5. AlarmTask

3.1 FreeRTOS Task Table

Task	Responsibility	Priority	Timing/Event
SensorTask	Reads environmental sensors and publishes sensor data	2	Periodic
DisplayTask	Updates the OLED display	1	Periodic/event-based

InputTask	Processes rotary encoder input	3	Periodic polling
MotionTask	Monitors PIR and manages activity state	3	Periodic polling
AlarmTask	Controls buzzer according to alarm event	2	Periodic/event-based

Higher priority values are assigned to InputTask and MotionTask because user input and motion detection require relatively short response times. SensorTask and AlarmTask use medium priority because their work is important but primarily periodic. DisplayTask has the lowest priority because display rendering can tolerate some scheduling delay.

3.2 Task States

The project demonstrates the three basic FreeRTOS task states:

- **Running** – the task currently executing.
- **Ready** – the task is able to execute but is waiting for CPU scheduling.
- **Blocked** – the task is waiting for a delay, event, queue, or synchronization condition.

Blocking delays are intentionally used to prevent tasks from continuously executing without giving other tasks opportunities to run.

3.3 Periodic Scheduling

SensorTask uses vTaskDelayUntil() for periodic sensor acquisition.

The use of vTaskDelayUntil() allows the task to maintain a periodic schedule based on a stored reference tick rather than simply delaying after each iteration. This is more appropriate for periodic sensor sampling because execution-time variations have less effect on the intended sampling period.

Other tasks use blocking vTaskDelay() calls where appropriate.

3.4 Inter-Task Communication

A FreeRTOS queue named sensorQueue is used to transfer SensorData between tasks.

SensorTask produces sensor information and places the structure into the queue. DisplayTask accesses the latest sensor information for display purposes.

The queue provides an explicit communication mechanism between tasks and avoids requiring every task to directly access the sensor hardware.

3.5 Event Group

The system uses a FreeRTOS event group with three event bits:

Event	Bit	Purpose
EVENT_ACTIVE	BIT0	Indicates ACTIVE system state
EVENT_MOTION	BIT1	Indicates detected motion
EVENT_ALARM	BIT2	Indicates an alarm condition

MotionTask manages the ACTIVE and MOTION events. Sensor/alarm logic manages the ALARM event. DisplayTask checks the ACTIVE event to determine whether the OLED should display information or remain blank. AlarmTask checks the ALARM event to determine whether the buzzer should be activated.

3.6 Mutex Synchronization

A FreeRTOS mutex named serialMutex protects shared Serial Monitor output.

Several tasks may produce diagnostic messages. Without synchronization, messages from different tasks can become interleaved. The mutex ensures that only one task accesses the protected serial-output section at a time.

3.7 Task Scheduling and Blocking

The tasks were deliberately designed to block regularly rather than continuously execute uncontrolled loops.

The fault experiments demonstrated the importance of this design. Removing blocking from the display task caused continuous OLED refreshing and increased CPU usage. Increasing the display task priority also affected the responsiveness of lower-priority tasks.

These observations support the use of explicit priorities and blocking delays in the final architecture.

4. Implementation

4.1 System Initialization

The ESP-IDF `app_main()` function performs the initial hardware and RTOS setup.

The initialization process includes:

1. Initializing the I2C interface.
2. Initializing the SSD1306 OLED.
3. Creating the sensor queue.

4. Creating the serial-output mutex.
5. Creating the system event group.
6. Setting the initial ACTIVE state.
7. Configuring the ADC for the LDR.
8. Initializing the DHT22.
9. Creating the five FreeRTOS application tasks.

The application uses ESP-IDF and native FreeRTOS APIs rather than the Arduino framework.

4.2 Sensor Implementation

SensorTask periodically obtains readings from the DHT22 and LDR.

The DHT22 provides:

- Temperature
- Humidity

The LDR is read through the ESP32 ADC. The resulting value is converted into a relative percentage used by the application.

The light value is therefore treated as a **relative light level**, not as a calibrated physical lux measurement.

4.3 DHT22 Error Handling

During development, intermittent invalid DHT22 readings were observed. Some readings produced physically implausible values, particularly for humidity.

To improve robustness, the sensor implementation was modified to validate temperature and humidity values before accepting them. If a reading fails or falls outside reasonable limits, the previously valid temperature and humidity values are retained.

This prevents one invalid sensor transaction from immediately propagating an obviously incorrect value through the system.

4.4 Display Implementation

DisplayTask is responsible for OLED access.

The selected display mode determines which value is shown:

- Temperature
- Humidity
- Light

- Motion

The OLED is therefore controlled by one task rather than being written to by several tasks. This provides clear ownership of the display resource.

When the system becomes INACTIVE, DisplayTask clears the OLED. When motion reactivates the system, the display becomes available again.

4.5 Rotary Encoder Implementation

The rotary encoder is used to navigate through the four display modes.

The logical sequence is:

Temperature → Humidity → Light → Motion → Temperature

Counterclockwise navigation reverses the sequence and also wraps around.

Using an enumeration for the display modes makes the navigation logic explicit and allows the forward and reverse transitions to be unit tested independently.

4.6 Alarm Implementation

The temperature alarm decision is separated from the physical buzzer control.

The alarm logic classifies a temperature as:

- LOW
- NORMAL
- HIGH

The result can therefore be tested independently of the ESP32 GPIO hardware.

AlarmTask monitors the EVENT_ALARM event. When the event is active, GPIO26 is driven to activate the buzzer. Otherwise, the buzzer output is turned off.

Threshold Issue Identified During Development

The laboratory requirement specifies:

- Low threshold: **18 °C**
- High threshold: **30 °C**

However, the development version of the alarm logic and its recorded unit tests used **20 °C** as the lower boundary.

This is an implementation discrepancy that must be corrected before final submission. The final source code and unit tests should use the laboratory-required 18 °C lower boundary.

4.7 Motion and Activity State

MotionTask monitors the PIR sensor.

When motion is detected:

- The motion status is updated.
- The last-motion time is refreshed.
- EVENT_MOTION is set.
- EVENT_ACTIVE is set.

If no motion occurs for the configured timeout, the system transitions to INACTIVE by clearing the corresponding active/motion state.

This design allows the motion-monitoring task to continue operating even while the system is INACTIVE, making reactivation possible when new motion is detected.

5. Verification and Testing

5.1 Unit Testing

The project contains 13 native unit tests covering three areas:

Test Category	Number of Tests	Purpose
Alarm logic	5	Tests temperature boundary behavior
Navigation	4	Tests encoder navigation and wraparound
System state	4	Tests ACTIVE/INACTIVE transitions
Total	13	

The recorded PlatformIO native test execution completed with **13/13 tests passing**.

The alarm tests included values below the lower threshold, at the lower boundary, within the normal range, at the upper boundary, and above the upper threshold.

However, because the development tests used a 20 °C lower threshold while the official requirement specifies 18 °C, the alarm tests must be updated and rerun after correcting the implementation.

The corrected boundary cases should be:

- 17.9 °C → LOW
- 18.0 °C → NORMAL
- 25.0 °C → NORMAL

- 30.0 °C → NORMAL
- 30.1 °C → HIGH

5.2 Wokwi Functional Testing

The following functional tests were recorded during Wokwi verification:

Test	Stimulus	Observed Result	Status
FT-01	Temperature changed to 52.9 °C	OLED displayed 52.9 °C	PASS
FT-02	Humidity changed to 69.0%	OLED displayed 69.0%	PASS
FT-03	Light input changed; simulator showed 3 lux	OLED displayed 91% relative light	PASS
FT-04	Rotary encoder turned clockwise	OLED changed to Motion and displayed YES	PASS
FT-05	Rotary encoder turned counterclockwise	OLED changed from Motion to Light	PASS
FT-06	Temperature changed to 35 °C	Expected alarm indication was not observed	FAIL
FT-07	Temperature returned to 24.0 °C	Alarm condition stopped	PASS
FT-08	PIR motion triggered	System became ACTIVE	PASS
FT-09	No motion until timeout	OLED became blank / system became INACTIVE	PASS
FT-10	PIR triggered while INACTIVE	OLED returned / system became ACTIVE	PASS

FT-06 is an important unresolved result. The expected alarm behavior was not observed when the temperature was raised to 35 °C. Therefore, the alarm function cannot be reported as completely verified based on the recorded test.

The light-level result is reported as a relative percentage because the implementation converts ADC input into a percentage. The Wokwi simulator's displayed lux value should not be interpreted as a calibrated lux measurement from the project.

01:32.483 62%

ROOM MONITOR
Temperature: 52.9 C

Problems Output Debug Console **Terminal** Ports ESP-IDF

ROOM MONITOR
Temperature: 52.9 C

00:36.242 84%

Editing DHT22

Temperature: 24.0°C

Humidity: 69.0%

ROOM MONITOR
Humidity: 69.0 %

Problems Output Debug Console **Terminal** Ports ESP-IDF

ROOM MONITOR
Humidity: 69.0 %

01:10.098 84%

Photoresistor (LDR)

ILLUMINATION (LUX)

3 lux

ROOM MONITOR
Light: 91 %

CLK DT SW + GND

ESP32

DHT22

PlatformIO ...
Wokwi Ter...

ROOM MONITOR
Light: 91 %

The screenshot shows a Wokwi simulation of an ESP32-based project. At the top, there are control buttons (refresh, stop, pause) and a timer showing 01:10.098 with 84% battery. A 'Photoresistor (LDR)' window is open, displaying a slider for 'ILLUMINATION (LUX)' set to 3 lux. Below this, the circuit is visible: an ESP32 microcontroller is connected to a DHT22 temperature and humidity sensor, a buzzer, and a photoresistor. The photoresistor is labeled 'ROOM MONITOR' and 'Light: 91 %'. The terminal at the bottom shows the output 'ROOM MONITOR Light: 91 %'.

ROOM MONITOR
Motion: YES

ESP32

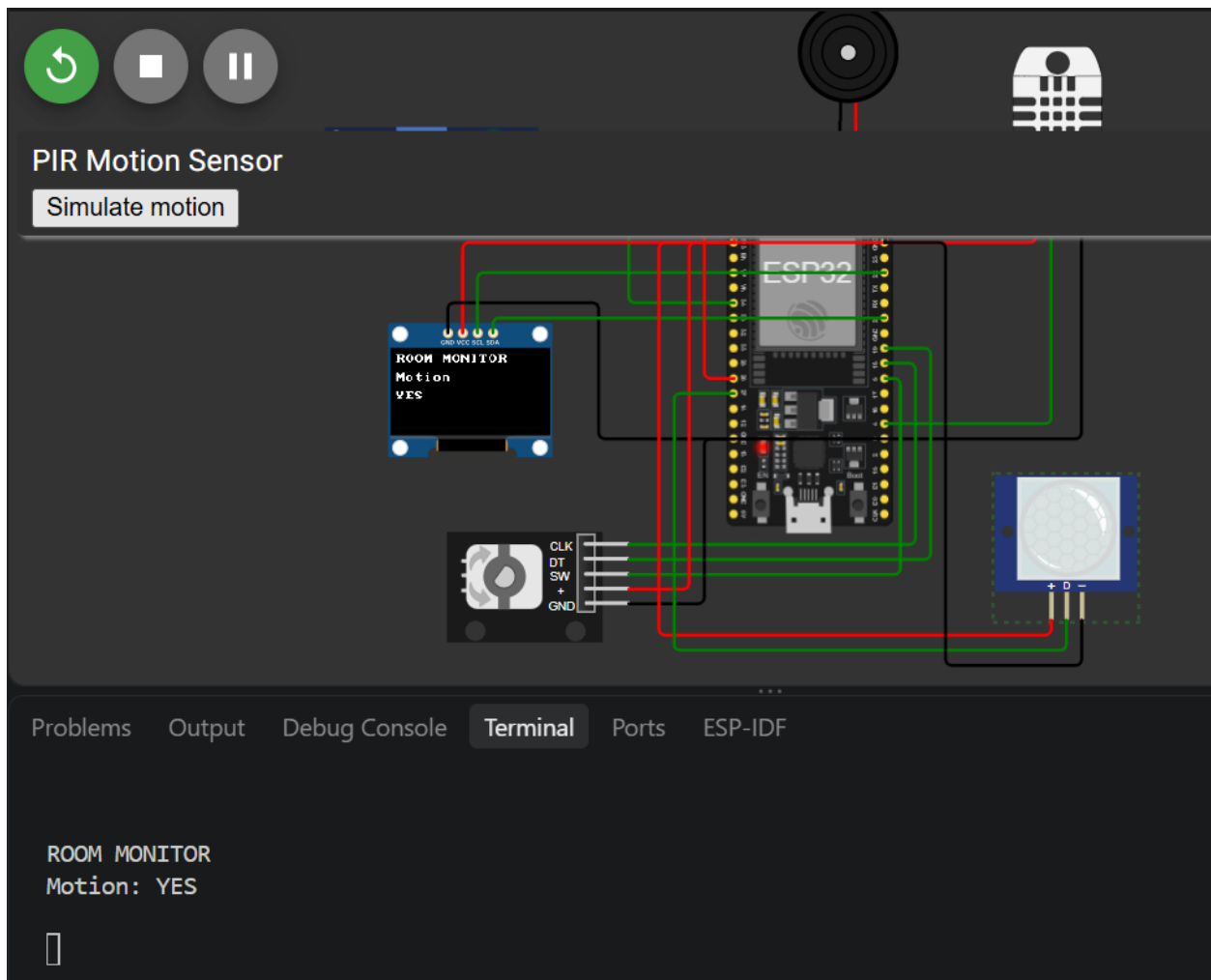
DHT22

ROOM MONITOR
Motion: YES

This screenshot shows the same Wokwi simulation but with a different sensor configuration. The photoresistor has been replaced with a motion sensor, also labeled 'ROOM MONITOR' and 'Motion: YES'. The ESP32 remains connected to the DHT22 sensor and the buzzer. The terminal output now reads 'ROOM MONITOR Motion: YES'.

ROOM MONITOR

Motion: NO



5.3 Deliberate Fault Experiment 1 — Removing Blocking

The blocking delay from the display task was temporarily removed.

The OLED then refreshed continuously, increasing processor activity and reducing the responsiveness of other tasks.

This demonstrated that a continuously executing task can consume excessive CPU time. The blocking delay was restored after the experiment.

5.4 Deliberate Fault Experiment 2 — Changing Task Priority

The DisplayTask priority was temporarily increased.

The display updated more aggressively, while lower-priority sensor, motion, and alarm tasks became less responsive.

The original priority was restored after the experiment.

This demonstrated that task priorities affect CPU scheduling and that unnecessarily high priority can interfere with other system functions.

5.5 Deliberate Fault Experiment 3 — Removing the Mutex

The serial-output mutex was temporarily removed.

Messages from multiple tasks became interleaved in the Serial Monitor, making the diagnostic output more difficult to interpret.

The mutex was restored after the experiment.

This demonstrated the importance of synchronization when multiple FreeRTOS tasks access a shared resource.

6. Static Code Analysis

PlatformIO static analysis was performed using the available Cppcheck integration.

6.1 Native Analysis

The native environment completed the static-analysis process and produced **nine low-severity unusedFunction findings**.

These findings involved functions that were not recognized as directly called by the static-analysis call graph.

Several of the affected functions were FreeRTOS task entry functions. Such functions are passed to `xTaskCreate()` and therefore may not appear as ordinary direct function calls in a static-analysis call graph.

Other findings involved helper functions that are used by the native unit-test environment.

6.2 Interpretation

The findings were reviewed rather than automatically deleting the affected functions.

For FreeRTOS task functions, the functions are intentionally retained because they serve as task entry points.

For test-related logic, functions are retained because they provide deterministic interfaces for native unit testing.

Therefore, the `unusedFunction` warnings were interpreted as limitations of the static-analysis call graph rather than automatic evidence that the functions should be removed.

6.3 ESP32 Analysis

The ESP32 environment's Cppcheck analysis did not complete successfully because the analysis tool/check itself failed.

This result was not interpreted as proof of a source-code defect. It indicates that the static-analysis process for that environment could not complete and should be rerun if the tool configuration is corrected.

6.4 Corrective Actions

The corrective approach was:

1. Review every reported function.
2. Determine whether the function is intentionally used by FreeRTOS task creation or unit testing.
3. Retain intentional functions.
4. Avoid deleting functions solely to eliminate a static-analysis warning.
5. Re-run analysis after significant source changes.
6. Record the ESP32 analysis-tool failure rather than presenting it as a successful clean analysis.

7. Engineering Discussion

7.1 FreeRTOS Task Decomposition

The project uses separate tasks because the system performs several activities concurrently.

A single-task implementation would be simpler initially, but it would make the system more difficult to schedule, test, and explain in terms of FreeRTOS concepts.

The five-task architecture provides clear responsibilities:

- SensorTask handles measurements.
- DisplayTask handles the OLED.
- InputTask handles the encoder.
- MotionTask handles PIR and activity state.

- AlarmTask handles the buzzer.

This decomposition also makes it possible to assign different priorities according to the responsiveness required by each function.

7.2 Scheduling Trade-offs

Task priority and execution frequency have direct effects on responsiveness and CPU usage.

Higher-priority tasks can respond sooner, but giving too many tasks high priority can reduce the CPU time available to lower-priority work.

Similarly, short delays can improve responsiveness but increase task wake-ups. Longer delays reduce processor activity but can increase response latency.

The fault experiments demonstrated these trade-offs directly.

7.3 Blocking Versus Busy Loops

The project intentionally uses blocking delays.

A task that continuously loops without blocking can consume excessive CPU time and interfere with other tasks. Blocking allows the FreeRTOS scheduler to execute other ready tasks.

The experiment in which the display delay was removed showed the practical effect of this issue.

7.4 Queue-Based Communication

The SensorData queue provides an explicit communication path between sensor acquisition and display processing.

This is preferable to having each task directly access sensor hardware because it separates acquisition from presentation.

The queue also demonstrates a core FreeRTOS inter-task communication mechanism required by the laboratory.

7.5 Event Groups

The event group provides a convenient way to represent system-wide conditions.

The three defined bits represent:

- ACTIVE
- MOTION
- ALARM

This makes system state and events available to the tasks that need them without requiring direct control of another task's internal implementation.

7.6 Shared Resource Protection

Serial output was selected as a practical shared resource for demonstrating mutex protection.

Without the mutex, messages from different tasks became interleaved. With the mutex, access to the protected output section is serialized.

This demonstrates that concurrency requires consideration of shared resources even when the shared resource is only diagnostic output.

7.7 Sensor Reliability

One of the development problems was intermittent invalid DHT22 readings.

The observed values included physically implausible humidity and temperature results. Accepting these values without validation could cause incorrect information to appear on the OLED or affect alarm decisions.

The implementation therefore validates readings and retains the previous valid values when an invalid reading occurs.

This approach improves robustness but does not completely solve sensor reliability. A production implementation could additionally use retries, explicit error reporting, sensor-health status, and hardware-level validation.

7.8 Alarm Verification Issue

The alarm test exposed an important implementation problem.

When the temperature was set to 35 °C, the expected alarm indication was not observed. The recorded result was therefore marked FAIL.

One contributing issue identified during development is the mismatch between the laboratory's specified lower threshold of 18 °C and the development implementation's lower threshold of 20 °C. Although this mismatch does not explain the high-temperature boundary itself, it demonstrates that the implementation and requirements were not yet completely synchronized.

The alarm path should therefore be reviewed and retested before final submission.

7.9 Wokwi Limitations

Wokwi provides useful simulation-based verification, but it does not reproduce every characteristic of physical hardware.

Physical implementation could introduce:

- Sensor noise
- Electrical timing differences
- Wiring problems

- Pull-up requirements
- ADC variation
- Actual sensor communication errors
- OLED hardware differences

Therefore, successful Wokwi testing should be considered simulation verification rather than complete physical-hardware validation.

7.10 Alternative Approaches

Several alternative approaches could be considered.

An event-driven approach could reduce unnecessary polling for the rotary encoder and PIR input. A dedicated StateTask could centralize ACTIVE/INACTIVE state management. More advanced queue designs could allow multiple sensor samples to be buffered instead of retaining only the most recent value.

The current design was selected because it clearly demonstrates the FreeRTOS concepts required by the laboratory while remaining understandable and testable.

7.11 Development and Debugging Experience

Several issues were encountered during development.

The native PlatformIO environment initially lacked the required GCC toolchain. MSYS2 UCRT64 GCC was installed and added to the Windows user PATH so that native tests could compile.

The native tests also required conditional handling of the application entry point so that the ESP-IDF `app_main()` would not conflict with the native Unity test runner.

The Wokwi simulation required a successful ESP32 firmware build before the firmware could be loaded into the simulator.

The DHT22 produced intermittent invalid readings, leading to the addition of input validation and retention of the last valid values.

These problems demonstrated that embedded development requires attention not only to application logic but also to build environments, simulation configuration, hardware interfaces, and test infrastructure.

8. Conclusion

The laboratory activity resulted in a modular ESP32-based multisensor room monitoring system using ESP-IDF and FreeRTOS. The project separates sensor acquisition, display management, rotary-encoder input, motion detection, and alarm control into independent tasks.

The implementation demonstrates the required FreeRTOS concepts, including multiple tasks, explicit priorities, blocking delays, periodic scheduling using `vTaskDelayUntil()`, queue-based communication, mutex synchronization, event groups, and an ACTIVE/INACTIVE state machine.

The recorded native testing produced 13/13 passing unit tests, covering alarm decision logic, navigation, and system-state behavior. Wokwi testing successfully demonstrated most of the required system functions, including sensor display, encoder navigation, motion detection, inactivity handling, and system reactivation.

However, FT-06, the high-temperature alarm activation test, did not produce the expected observed result and therefore remains a failed functional test in the recorded verification. In addition, the development implementation used a 20 °C low-temperature boundary while the laboratory requirement specifies 18 °C. These issues must be corrected and retested before the final submission can be considered fully verified.

The project demonstrated that successful embedded-system development requires more than simply making the hardware simulation work. Task decomposition, scheduling, synchronization, inter-task communication, deterministic testing, static-analysis interpretation, debugging, and documentation are all important parts of a reliable real-time system.

Future improvements should include correcting and retesting the alarm threshold and alarm output path, performing additional integration testing, improving sensor error handling, calibrating the light sensor on physical hardware, and repeating the complete verification process using an actual ESP32 and physical sensors.

FreeRTOS Task Summary

Task	Priority	Main Function	Communication / Synchronization
SensorTask	2	Sensor acquisition	Queue, Event Group, Mutex
DisplayTask	1	OLED display	Queue, Event Group
InputTask	3	Rotary encoder	Event Group, Mutex
MotionTask	3	PIR and activity state	Event Group
AlarmTask	2	Buzzer control	Event Group

Requirements Traceability Matrix

Requirement	Implementation	Verification
FR-01 Temperature	DHT22 handled by SensorTask	FT-01
FR-02 Humidity	DHT22 handled by SensorTask	FT-02
FR-03 Ambient Light	LDR through ESP32 ADC	FT-03
FR-04 Motion	PIR handled by MotionTask	FT-08, FT-10
FR-05 OLED Display	DisplayTask / SSD1306	FT-01–FT-05, FT-08–FT-10
FR-06 Navigation	InputTask / DisplayMode	Navigation unit tests, FT-04, FT-05
FR-07 Temperature Alarm	Alarm evaluation, EVENT_ALARM, AlarmTask	Alarm unit tests, FT-06, FT-07
FR-08 ACTIVE/INACTIVE	MotionTask and event group	System-state unit tests
FR-09 Inactivity	MotionTask timeout	FT-09
FR-10 Reactivation	PIR → ACTIVE event	FT-10